

11420 AIA 5350 Deep Learning

Lab 6

111061220 林進成

1. Introduction

Generative modeling aims to learn the underlying distribution of a dataset and synthesize new samples that are visually and semantically consistent with the training data. In this lab, the target task is conditional image generation on the i-CLEVR dataset, where each image may contain multiple-colored geometric objects. Given a set of object labels such as “red sphere”, “cyan cylinder”, and “cyan cube”, the model is expected to generate a 64×64 RGB image containing the specified objects.

This project implements a conditional Denoising Diffusion Probabilistic Model (DDPM). A DDPM defines a forward diffusion process that gradually corrupts a clean image by adding Gaussian noise over a sequence of timesteps and learns a reverse denoising process that reconstructs an image from pure noise. During training, the model receives a noisy image, a timestep, and a multi-label condition vector, then predicts the noise added to the original image. The denoising objective is optimized using a mean squared error loss between the predicted and sampled noise.

To support conditional generation, each object label set is converted into a 24-dimensional multi-hot vector according to the i-CLEVR object dictionary. This condition vector is projected into an embedding space and injected into the U-Net denoising network as class conditioning. Therefore, the reverse diffusion process is guided not only by the current noisy image and timestep, but also by the desired object composition.

The final model is evaluated using the pretrained classifier provided by the assignment. Generated images for both “test.json” and “new_test.json” are passed into the evaluator together with their ground-truth multi-hot labels, and the classification accuracy is used as the quantitative metric. In addition, image grids and a denoising process visualization are produced to qualitatively demonstrate whether the model can generate images that match the given multi-label conditions.

2. Implementation

2.1 Dataset preprocessing and condition representation

The task is formulated as multi-label conditional image generation. Each training sample contains an i-CLEVR image and a list of object labels, while each testing sample contains only a target label list. According to the dataset specification, i-CLEVR contains 24 object categories formed by different colors and shapes, and each image may contain one to three objects. The testing files “test.json” and “new_test.json” each contain 32 label conditions.

In the implementation, the training dataset reads image-label pairs from “train.json”, loads the corresponding RGB image from the data/ directory, resizes it to 64×64 , converts it to a tensor, and normalizes it to the range expected by the evaluator using:

```
transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
```

Each label list is converted into a 24-dimensional multi-hot vector. For example, if the target condition is ["red sphere", "cyan cylinder"], the indices corresponding to these two objects are set to 1 and all other entries remain 0. This representation is used for both training and testing. The repository implements this behavior in `iCLEVRDataset`, `TestConditionDataset`, and `labels_to_multihot`.

```
def labels_to_multihot(labels, objects_map, num_classes):
    multihot = torch.zeros(num_classes, dtype=torch.float32)
    for label in labels:
        multihot[objects_map[label]] = 1.0
    return multihot
```

This design is necessary because the condition is not a single class index but a set of multiple objects. Therefore, a standard class embedding table is not directly suitable; instead, the condition must be represented as a multi-label vector.

2.2 Conditional DDPM formulation

The model is implemented as a conditional Denoising Diffusion Probabilistic Model (DDPM). The forward diffusion process gradually adds Gaussian noise to a clean image x_0 over T timesteps:

$$q(x_t | x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t}x_0, (1 - \bar{\alpha}_t)I)$$

where t is the diffusion timestep, $\bar{\alpha}_t$ is the cumulative product of the noise schedule coefficients, and x_t is the noisy image. During training, a random timestep t is sampled for each image, Gaussian noise ϵ is added, and the neural network is trained to predict the added noise.

The reverse process starts from pure Gaussian noise and iteratively removes noise using the learned denoising model:

$$p_\theta(x_{t-1} | x_t, y)$$

where y is the multi-label condition vector. In this lab, y specifies which objects should appear in the generated i-CLEVR image. This follows the DDPM setting in which the model predicts noise $\epsilon_\theta(x_t, t, y)$ rather than directly predicting the clean image. The `DDPMScheduler` also supports `prediction_type="epsilon"`, which corresponds to predicting the diffusion noise.

The training objective is:

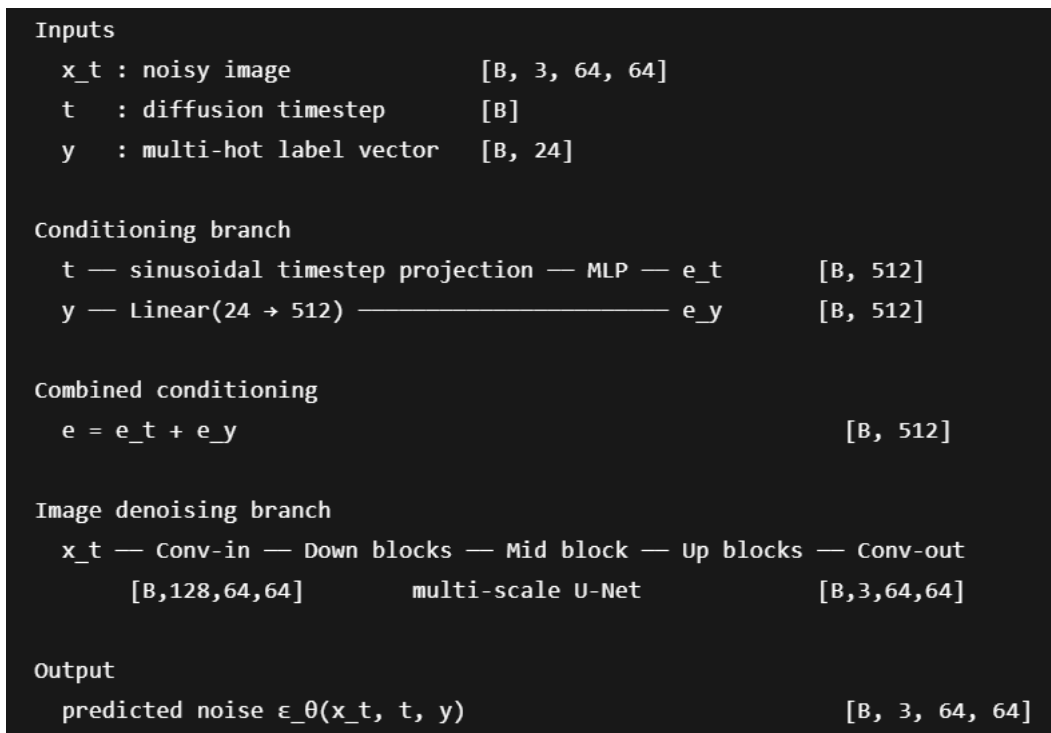
$$\mathcal{L}_{noise} = \mathbb{E}_{x_0, t, \epsilon, y} [\| \epsilon - \epsilon_\theta(x_t, t, y) \|_2^2]$$

This is implemented as mean squared error between the sampled Gaussian noise and the model-predicted noise.

2.3 Model architecture

The denoising network is implemented as a conditional U-Net based on Hugging Face Diffusers' `UNet2DModel`. The model takes three inputs: the noisy image x_t , the diffusion timestep t , and the multi-label object condition y . The output has the same spatial size as the input image and represents the predicted Gaussian noise $\epsilon_\theta(x_t, t, y)$. In the repository, this is implemented by the `ConditionalDDPM` class, which wraps a `UNet2DModel` and a separate label embedding layer.

The overall architecture can be decomposed into two branches: the image denoising branch and the conditioning branch. The image branch processes the noisy image through the U-Net encoder-decoder structure, while the conditioning branch converts the timestep and multi-hot label vector into a shared embedding that is injected into the U-Net ResNet blocks.



The label condition is represented as a 24-dimensional multi-hot vector because each i-CLEVR image may contain multiple objects. Instead of using a standard embedding table for single-class labels, the implementation uses a linear projection:

$$e_y = W_y y + b_y, y \in \mathbb{R}^{24}, e_y \in \mathbb{R}^{512}$$

This is implemented by the `ConditionEmbedder` module:

```
class_embeds = self.condition_embedder(labels_multihot)
noise_pred = self.unet(
    noisy_images,
    timesteps,
    class_labels=class_embeds,
).sample
```

The model uses `class_embed_type="identity"` in `UNet2DModel`, so the 512-dimensional label embedding is directly treated as the class embedding. Diffusers' `UNet2DModel` sums the class embedding with the timestep embedding when class conditioning is provided. Therefore, the combined embedding $e = e_t + e_y$ is used as the conditioning signal for the U-Net.

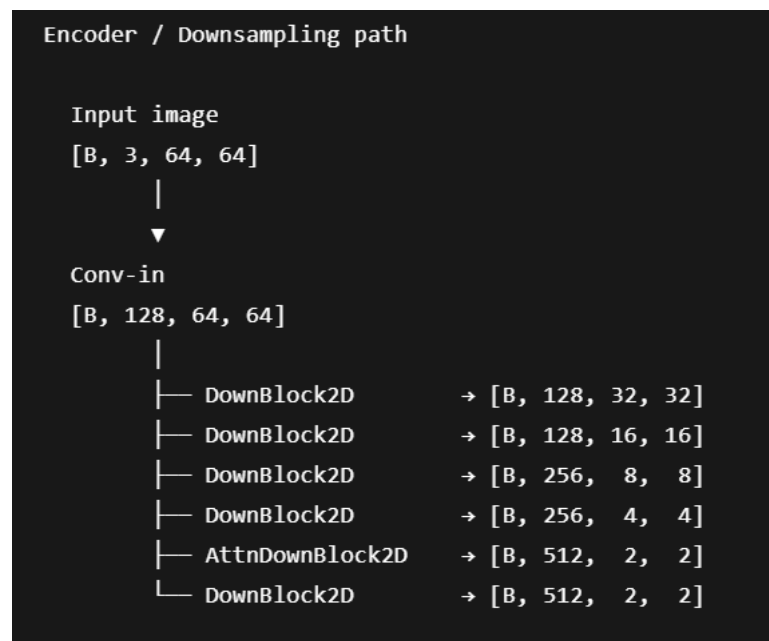
Inside the U-Net, this combined embedding is supplied to the ResNet blocks in the downsampling path, middle block, and upsampling path. For each ResNet block, the embedding $e \in \mathbb{R}^{B \times 512}$ is projected to match the channel dimension of that block, reshaped to $[B, C, 1, 1]$, and broadcast over the spatial feature map. As a result, the timestep and label information modulates the feature transformation at every resolution level:

For a ResNet block with feature map h :

h	$[B, C, H, W]$
$e = e_t + e_y$	$[B, 512]$
<code>Linear(e)</code>	$[B, C]$
<code>reshape / broadcast</code>	$[B, C, 1, 1]$
$h + \text{projected embedding}$	$[B, C, H, W]$

This design allows the same U-Net to learn different denoising behavior for different timesteps and object conditions. The timestep embedding tells the model how much noise is present, while the label embedding tells the model what objects should be generated.

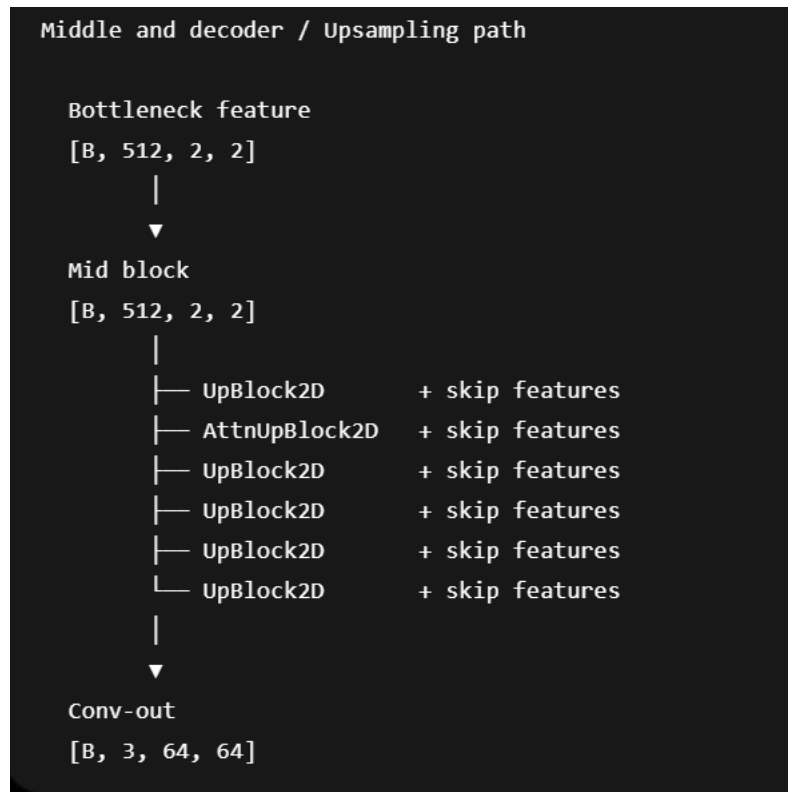
The image branch follows an encoder-decoder U-Net structure. The encoder progressively reduces spatial resolution and increases the channel dimension, allowing the model to extract high-level features from the noisy image. The decoder then upsamples the feature maps and combines them with skip connections from the encoder, preserving spatial details that are useful for reconstructing object shapes and positions.



The model uses six channel stages:

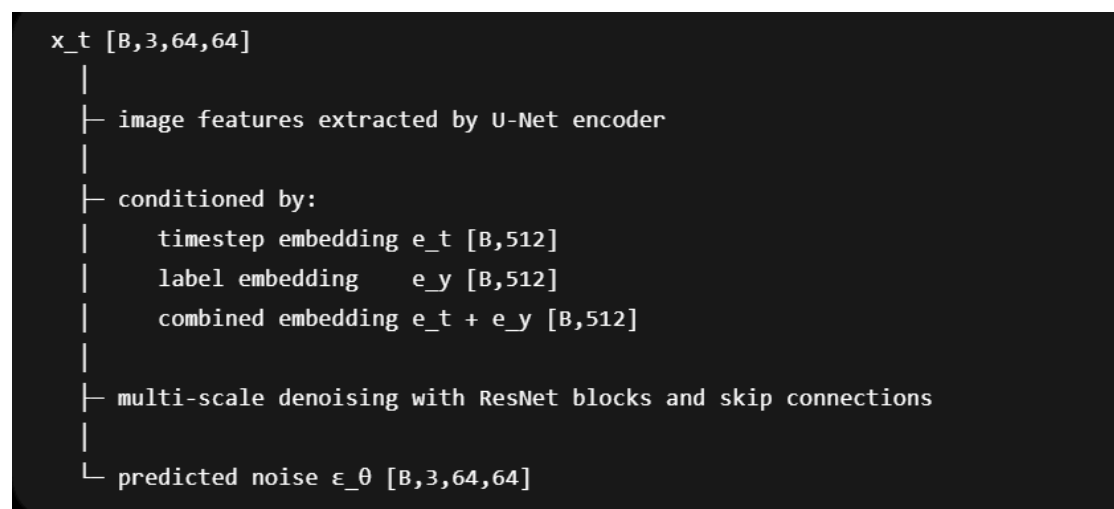
(128, 128, 256, 256, 512, 512)

Each block contains two ResNet layers. The attention block in the deeper encoder stage allows the network to model relationships between spatial locations after the feature map has been compressed. This is useful for multi-object generation because the model must place several objects in one image while keeping their shapes and colors consistent.



The skip connections pass encoder features to the corresponding decoder stages. This is important because pure bottleneck features may lose fine spatial information, while the final generated image still needs clear object boundaries and recognizable shapes. The decoder therefore uses both global semantic features from the bottleneck and local details from the encoder.

Conceptually, one denoising forward pass can be summarized as:



This architecture is suitable for the assignment because it directly matches the DDPM noise-prediction objective. During training, the model learns to predict the noise added to an image at a randomly sampled timestep. During inference, the same model is repeatedly applied in the reverse diffusion process to gradually transform Gaussian noise into an image that matches the given multi-label condition. The generated images and denoising trajectory are presented in the next section rather than here, since they are experimental results rather than architectural components.

2.4 Noise schedule

The diffusion process uses a discrete DDPM scheduler with 1000 training timesteps. The implementation uses Hugging Face Diffusers' `DDPMScheduler` with:

```
DDPMScheduler(  
    num_train_timesteps=1000,  
    beta_schedule="squaredcos_cap_v2",  
    prediction_type="epsilon",  
)
```

The `squaredcos_cap_v2` schedule is selected instead of a simple linear schedule because it changes the signal-to-noise ratio more smoothly across timesteps. In early timesteps, the clean image signal remains relatively strong, while in later timesteps the noise gradually dominates. This makes the denoising problem more stable across different noise levels. The Diffusers documentation lists `squaredcos_cap_v2` as one of the supported beta schedules for `DDPMScheduler`, and `add_noise` is used to add noise to original samples according to the timestep-specific noise magnitude.

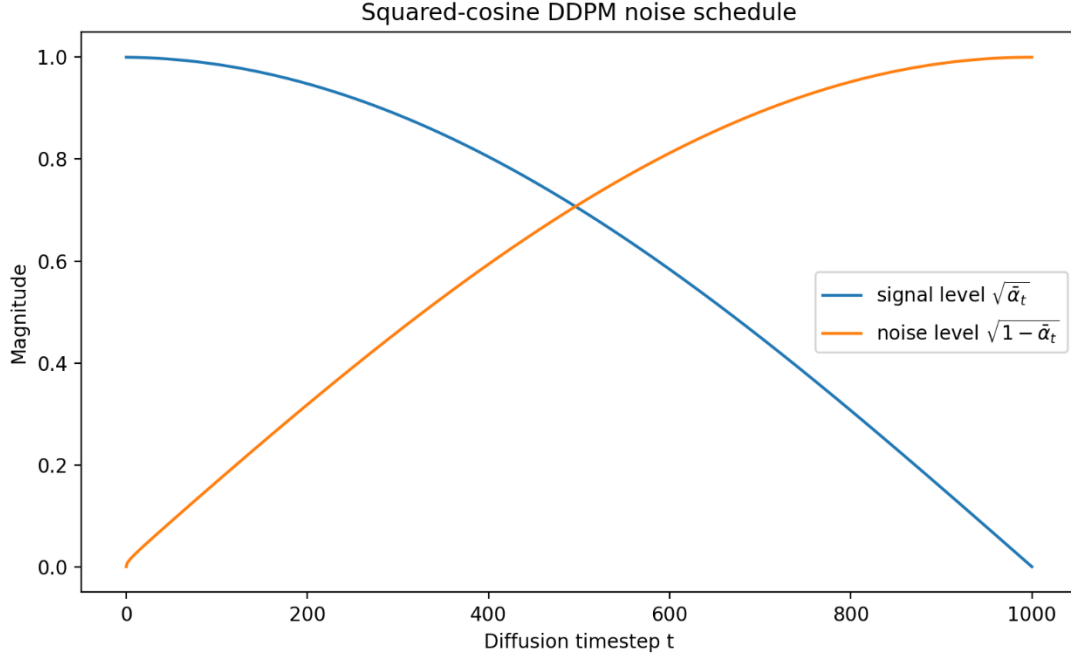


Figure 1. Squared-cosine DDPM noise schedule.

The plotted signal level $\sqrt{\bar{\alpha}_t}$ decreases from approximately 1 to 0, while the noise level $\sqrt{1 - \bar{\alpha}_t}$ increases from approximately 0 to 1. This illustrates how the forward process gradually converts a clean image into Gaussian noise.

2.5 Training procedure

For each training iteration, a batch of normalized images and multi-hot labels is sampled from the dataloader. A random timestep is sampled independently for each image. Gaussian noise is then sampled and added to the clean image using the DDPM scheduler. The model predicts the injected noise from the noisy image, timestep, and condition vector. The main loss is the MSE noise-prediction loss. This logic is implemented in the training script.

```

noise = torch.randn_like(images)
timesteps = torch.randint(
    0,
    scheduler.config.num_train_timesteps,
    (batch_size,),
    device=device,
    dtype=torch.long,
)
noisy_images = scheduler.add_noise(images, noise, timesteps)

predicted_noise = model(noisy_images, timesteps, labels)
noise_loss = F.mse_loss(predicted_noise, noise)

```

The optimizer is AdamW. Gradient clipping is applied to prevent unstable updates, and gradient accumulation is supported to increase the effective batch size when GPU memory is limited. The current training script also uses a OneCycle learning-rate schedule, with an initial learning rate, maximum learning rate, and minimum learning rate configured through command-line arguments. This is used to warm up the optimizer early in training and reduce the learning rate later for more stable convergence.

```

lr_scheduler = OneCycleLR(
    optimizer,
    max_lr=config.onecycle_max_lr,
    total_steps=total_optimizer_steps,
    pct_start=config.onecycle_pct_start,
    anneal_strategy="cos",
    cycle_momentum=False,
    div_factor=div_factor,
    final_div_factor=final_div_factor,
)

```

In addition to the noise-prediction loss, the implementation includes an optional evaluator-guidance loss. The provided evaluator is a pretrained ResNet18 classifier that outputs object probabilities for generated i-CLEVR images. The assignment allows using this evaluator for classifier guidance, as long as the evaluator script and weights are not modified.

The auxiliary guidance branch reconstructs an estimate of the clean image $\hat{x}_0$ from x_t and the predicted noise, feeds it into the frozen evaluator, and computes binary cross-entropy against the multi-hot label. The total loss is:

$$\mathcal{L} = \mathcal{L}_{noise} + \lambda \mathcal{L}_{guidance}$$

where λ is the evaluator-guidance weight. This auxiliary term encourages the reconstructed image estimate to be classified as containing the target objects. However, the main optimization target remains the DDPM noise-prediction objective.

```
x0_hat = reconstruct_x0(noisy_images, predicted_noise, timesteps, scheduler)
evaluator_pred = evaluator_net(x0_hat)
guidance_loss = F.binary_cross_entropy(evaluator_pred, labels)
total_loss = noise_loss + config.evaluator_guidance_weight * guidance_loss
```

2.6 Sampling and testing procedure

During inference, the model starts from a random Gaussian tensor with shape $[B, 3, 64, 64]$. The DDPM scheduler iterates through the reverse timesteps. At each timestep, the conditional U-Net predicts the noise component, and the scheduler updates the current sample to the previous timestep:

$$x_t \rightarrow x_{t-1}$$

This process is repeated until the final image x_0 is obtained. The repository implements this in `sample_images`, using the same scheduler configuration as training.

```
samples = torch.randn(batch_size, 3, image_size, image_size, device=device)

for timestep in scheduler.timesteps:
    timestep_batch = torch.full(
        (batch_size,),
        int(timestep),
        device=device,
        dtype=torch.long,
    )
    noise_pred = model(samples, timestep_batch, conditions)
    samples = scheduler.step(noise_pred, timestep, samples).prev_sample
```

For evaluation, the model generates images for both “test.json” and “new_test.json”. The generated tensors remain normalized when passed into the evaluator, while denormalized versions are saved as PNG images for visualization. The script saves individual images following the order of the condition files and also creates grid images with 8 images per row.

The denoising-process visualization is generated using the required label set “"red sphere", "cyan cylinder", "cyan cube"”. Intermediate samples are saved during the reverse diffusion process and arranged into a single-row grid. The generated image grids, denoising visualization, and quantitative accuracies are reported in the next section, “Results and Discussion,” to avoid mixing implementation details with experimental outcomes.

3. Results and Discussion

3.1 Results

The trained conditional DDPM was evaluated on both official testing condition files, test.json and new_test.json. For each condition entry, the model generated one 64×64 RGB image from Gaussian noise through the reverse DDPM sampling process. The generated images were then evaluated by the TA-provided pretrained classifier.

The final evaluation accuracies are:

test.json accuracy	new_test.json accuracy
0.8066	0.8333

Table 1. The evaluation results of “test.json” and “new_test.json”

These results show that the model can generate images that match most of the given multi-label object conditions. Both testing splits exceed 0.8 accuracy, which indicates that the model learned not only the basic appearance of i-CLEVR objects, but also the conditional relationship between multi-hot labels and generated image content.

Training loss

The training curve of “train/total_loss_step” shows a clear decreasing trend throughout training. The loss drops rapidly in the early stage and then continues to decrease more gradually. Although several spikes appear during training, the overall trend remains stable and downward. This behavior is expected in DDPM training because each step samples random timesteps and random Gaussian noise, causing batch-level loss



fluctuation.

Figure 2. Training curve of “train/total_loss_step”.

Synthetic image grids

The generated image grids for “test.json” and “new_test.json” are shown below. Each grid contains 32 generated images arranged in 4 rows and 8 columns, following the order of the corresponding JSON file.

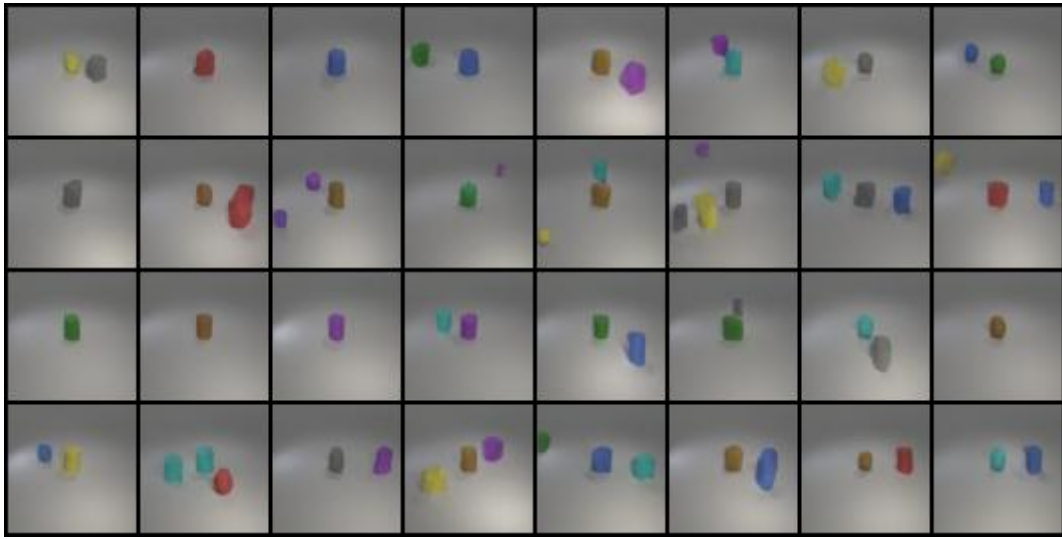


Figure 3. Generated images on “test.json”.



Figure 4. Generated images on “new_test.json”

From visual inspection, most generated samples contain clear colored geometric objects such as cubes, spheres, and cylinders. The generated objects generally have recognizable colors and shapes, and many samples contain multiple objects as required by the input conditions. Some images still show imperfect object boundaries or ambiguous shapes, especially when multiple objects overlap or appear close to each

other. However, the overall visual quality is sufficient for the evaluator to identify most target labels correctly.

Denoising process visualization

To visualize the reverse diffusion process, I generated a denoising sequence using the required label condition “"red sphere", "cyan cylinder", "cyan cube"”.

The denoising process starts from pure Gaussian noise and gradually forms a structured image. In the early steps, the image is dominated by random noise. As the reverse process proceeds, object colors and rough positions begin to appear. In the final step, the generated image contains recognizable red and cyan objects, demonstrating that the model progressively transforms noise into a condition-matching image.

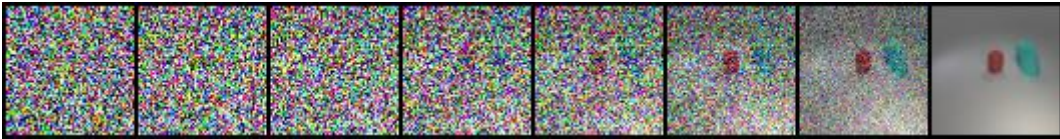


Figure 5 here: Denoising process for ["red sphere", "cyan cylinder", "cyan cube"].

Overall, the quantitative results and generated samples show that the implemented conditional DDPM successfully performs multi-label image generation on i-CLEVR. The model achieves 0.8066 accuracy on “test.json” and 0.8333 accuracy on “new_test.json”, and the visual results confirm that the generated images are mostly consistent with the requested object conditions.

3.2 Discussion

Although the final model achieves acceptable performance on both testing splits, the experimental results also show that checkpoint selection is important for this task. In this experiment, I selected the checkpoint from **epoch 98** for final evaluation. This checkpoint achieved **0.8066** accuracy on test.json and **0.8333** accuracy on

new_test.json. However, the later checkpoints from epoch 99 and epoch 100 did not satisfy the required accuracy threshold of 0.8 on both testing files. This indicates that a later checkpoint does not necessarily guarantee better generation quality.

One possible reason is that the training objective and the final evaluation metric are not exactly the same. During training, the model mainly optimizes the DDPM noise-prediction objective, while the final score is computed by a pretrained classifier on generated samples. Therefore, a lower training loss does not always lead to a higher evaluator accuracy. In addition, the sampling process itself is stochastic because each generated image starts from random Gaussian noise. As a result, the final evaluator accuracy may fluctuate between checkpoints, even when the training loss continues to decrease.

From the training curve, the total loss still shows a decreasing trend and has not clearly reached a plateau. This suggests that the model may still have room for improvement if trained longer or tuned more carefully. However, since the current checkpoint already satisfies the required accuracy threshold, I chose not to spend additional training time pursuing marginal improvements. Further improvements may require more systematic experiments, such as longer training, different learning-rate schedules, lower guidance weight, multiple candidate sampling, or evaluator-based reranking during inference.

The generated image grids show that the model has learned the general visual style of i-CLEVR images. Most samples contain recognizable colored objects, and many of them correctly match the requested object conditions. However, some generated objects still have imperfect boundaries, ambiguous shapes, or missing objects. These errors are more likely to appear in multi-object conditions, where the model must generate several objects with correct colors and shapes simultaneously.

Another limitation is that the model is still sensitive to the random seed during sampling. Before the model reaches ideal conditional generation performance, changing only the sampling seed can lead to noticeably different results for the same label condition. For example, using the condition ["red sphere", "cyan cylinder", "cyan cube"], one generated denoising result successfully produced the target objects, but another seed produced an incorrect sample where the **red sphere was missing**.

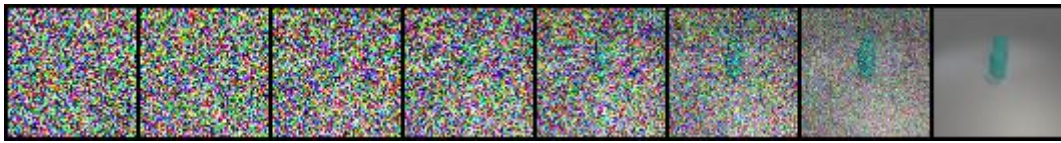


Figure 6 here: Failed denoising example caused by sampling randomness.

This observation suggests that the model has learned a useful conditional mapping, but the mapping is not fully robust. A practical way to reduce this issue would be to generate multiple candidate images for each condition and select the best one using the pretrained evaluator. This would not change the model itself, but it could make the final testing result more stable. Another possible improvement is to continue training with a more stable learning-rate schedule or reduce the influence of high-variance guidance loss during the later training stage.

Overall, the model successfully meets the main objective of the assignment by generating conditional i-CLEVR images with accuracy above 0.8 on both test.json and new_test.json. Nevertheless, the results also show that the current model is not fully converged and remains sensitive to checkpoint choice and sampling seed. These limitations explain why the final checkpoint was selected based on actual evaluator accuracy rather than simply choosing the last training epoch.